



Diagrama de Classes

O diagrama de classes é, sem dúvida, a ferramenta mais importante da **UML (Unified Modeling Language)**. Se a Orientação a Objetos (OO) fosse a construção de um prédio, o diagrama de classes seria a **planta baixa**.

Utilizá-lo é fundamental por vários motivos que vão além de "apenas documentar":

1. Visualização da Estrutura do Sistema

É difícil entender a relação entre centenas de arquivos de código apenas lendo o texto. O diagrama permite ver, em uma única imagem, como as classes se conectam (quem "conhece" quem, quem "herda" de quem).

Diagrama de Classes

2. Definição de Responsabilidades

No diagrama, você define os **Atributos** (o que a classe sabe) e os **Métodos** (o que a classe faz). Isso ajuda a evitar que uma classe fique com funções demais (a famosa "Classe Deus"), mantendo o princípio de responsabilidade única.

3. Padronização da Comunicação

O diagrama de classe é uma linguagem universal. Um desenvolvedor no Brasil e um na China podem entender a arquitetura de um software sem precisar falar o mesmo idioma, apenas seguindo as regras da UML.

4. Identificação de Erros Antes do Código

É muito mais barato apagar uma linha no diagrama do que refatorar 2.000 linhas de código. Ao desenhar, você percebe falhas de lógica, como:

- Dependências circulares.
- Falta de atributos necessários.
- Acoplamento excessivo (quando uma classe depende demais de outra).

Diagrama de Classes

5. Documentação e Manutenção

Sistemas mudam. Quando um novo desenvolvedor entra na equipe, o diagrama de classes serve como um mapa para que ele entenda onde cada peça do "quebra-cabeça" se encaixa sem precisar explorar todo o repositório.

Classificação – Conceitos de O.O



Nós aprendemos conceitos simples, tais como pessoa, carro e casa, por exemplo, e ao fazerem isso, definem **classes** ou seja **grupo** de objetos, sendo que cada **objeto** é um exemplo de **um determinado grupo**, tendo as mesmas **características** e **comportamentos de qualquer objeto** do grupo em questão

Abstração – Conceitos de O.O



A abstração nos permite lidar com a complexidade do mundo real focando apenas no que é essencial. Veja como aplicamos isso ao conceito de **Carro**:

- **Observação da Diversidade:** Embora os carros apresentem diferentes formatos, cores e estilos, todos compartilham propriedades fundamentais.
- **Identificação de Padrões:** Cada objeto-carro possui características semelhantes, como rodas, portas, faróis e a capacidade de transportar pessoas de um lugar para outro.
- **O Ato de Abstrair:** Ao ignorar as diferenças (como a cor específica) e focar nessas semelhanças, "abstraímos" uma **Classe**: o molde conceitual chamado "Carro".
- **Reconhecimento de Instâncias:** Sempre que identificamos um objeto real com essas características pré-determinadas, concluímos que ele é uma **instância** (um exemplo concreto) da classe Carro.

Instanciar – Conceitos de O.O



É o ato de criar um novo item a partir do molde definido por uma classe. Esse novo objeto herda automaticamente a estrutura de características e comportamentos do conjunto ao qual pertence.



Embora compartilhem a mesma estrutura (atributos), os objetos não são idênticos. Cada instância armazena seus próprios **valores** específicos em seus atributos (ex: um carro pode ser azul e o outro vermelho), garantindo a individualidade de cada objeto no sistema.

Pq é Importante ?

1. Abstração: O "Filtro" do Diagrama

A abstração impede que seu diagrama fique poluído com detalhes desnecessários. Ela te ajuda a definir o **escopo**.

- ✓ **Como ajuda:** Se você está criando um sistema de vendas, a abstração te diz que a classe Carro precisa do atributo preço, mas não precisa do atributo pressão do óleo.
- ✓ **No Diagrama:** Resulta em classes com atributos e métodos enxutos e focados no problema que você quer resolver.

► Pq é Importante ?

2. Classificação: A "Estrutura" do Diagrama

A classificação é o ato de organizar a bagunça. Ela ajuda a identificar quais "moldes" (classes) precisam existir no seu projeto.

- ✓ **Como ajuda:** Ao observar que vários elementos têm características semelhantes, você cria uma única caixa de Classe em vez de várias caixas redundantes.
- ✓ **No Diagrama:** Define o nome da classe e o agrupamento lógico de atributos e comportamentos que todos os objetos daquele grupo terão.

Pq é importante ?

3. Instanciação: A "Validação" do Diagrama

Embora o diagrama de classe não mostre os objetos individuais (instâncias), entender o instanciamento ajuda a testar se o seu diagrama faz sentido na prática.

- ✓ **Como ajuda:** Você mentaliza: "Se eu instanciar um objeto dessa classe agora, eu consigo preencher todos os dados necessários?".
- ✓ **No Diagrama:** Ajuda a definir a **multiplicidade** (ex: um Cliente pode ter várias instâncias de Pedido) e a garantir que cada instância terá valores únicos em seus atributos, mesmo compartilhando a mesma estrutura.

Diagrama de Classes

"O Diagrama de Classes é reconhecido como o recurso mais importante e amplamente utilizado da **UML**. Suas principais funções no desenvolvimento de sistemas são:

- ✓ **Visualização Estrutural:** Permite detalhar as classes do sistema, seus respectivos atributos, métodos e os relacionamentos entre elas.
- ✓ **Visão Estática:** Apresenta a organização das classes de forma fixa, sem considerar a dinâmica temporal das operações.
- ✓ **Abstração Lógica:** Foca exclusivamente na estrutura lógica da solução.
- ✓ **Fundamentação de Projeto:** Serve como base indispensável para a criação de outros diagramas da UML."

Diagrama de Classes



- **Persistência de Dados:** Em diversos cenários, é fundamental preservar o estado dos objetos de uma classe de forma permanente.
- **Mapeamento de Entidades:** Uma classe possui fortes semelhanças conceituais com uma **entidade** do Modelo Entidade-Relacionamento (MER).
- **Modelagem de Banco de Dados:** Devido a essa similaridade, as classes são frequentemente utilizadas como base para definir a estrutura das tabelas em bancos de dados relacionais.

Classe de Objetos



Uma classe representa uma categoria e os objetos são membros ou exemplos dessa categoria. Na UML classe é representada por um retângulo que pode ter até três divisões.

A primeira divisão armazena o nome pelo qual a classe é identificada,

Atributos

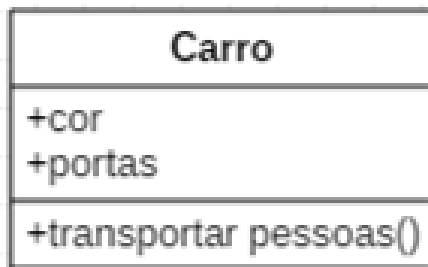


Os atributos são apresentados na segunda divisão da classe e contêm, normalmente, duas informações: o nome que identifica o atributo e o tipo de dado que o atributo armazena, como por exemplo, integer, float ou character. Essa informação não é obrigatória, mas é útil e recomendável.

Na realidade, não é exatamente a classe que contém os atributos, mas sim os objetos dessa classe.

Os objetos têm os atributos relativos à classe à qual pertencem. Tais atributos são as características do objeto.

Métodos



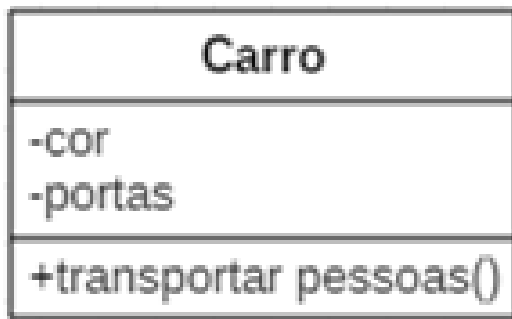
Classes costumam ter métodos, também conhecidos como operações ou comportamentos.

Um método representa um conjunto de instruções que são executadas quando o método é chamado, por exemplo, um objeto da classe Carro pode executar a atividade de transportar pessoas.

Grande parte da codificação propriamente dita dos sistemas de informação orientado a objeto está contida nos métodos definidos em suas classe.

Os métodos são armazenados na terceira divisão de uma classe.

Visibilidade



A visibilidade define quem pode "enxergar" e interagir com o que está dentro de uma classe.

O objetivo principal é proteger a integridade dos dados, impedindo que informações sensíveis sejam alteradas indevidamente por agentes externos.

No Diagrama de Classes da UML, a visibilidade é representada por símbolos específicos antes do nome do atributo ou método:

Visibilidade

Carro
-cor -portas
+transportar pessoas()

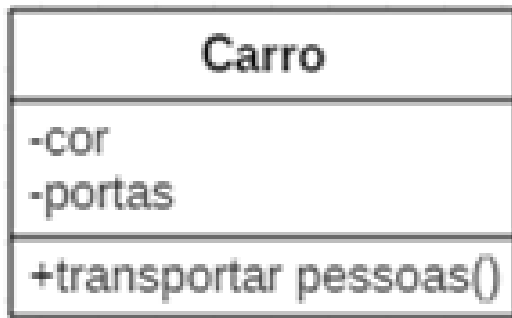
1. Pública (+)

- **Descrição:** É o nível de acesso mais aberto. Os membros marcados como públicos podem ser acessados livremente por qualquer outra classe, independente do pacote ou hierarquia.
- **Uso Comum:** Geralmente aplicada aos métodos da classe (como os métodos "Get" e "Set") para permitir que o mundo externo interaja com o objeto.

2. Privada (-)

- **Descrição:** É o nível mais restrito. Apenas a própria classe que definiu o membro pode acessá-lo ou modificá-lo.
- **Uso Comum:** Essencial para o **Encapsulamento**. Quase todos os atributos devem ser privados para evitar que dados sensíveis sejam alterados sem validação.

Visibilidade



3. Protegida (#)

- **Descrição:** Oferece um meio-termo. Os membros são acessíveis pela própria classe e por todas as suas **subclasses** (classes que herdam dela), mas permanecem privados para o resto do sistema.
- **Uso Comum:** Utilizada em hierarquias de herança para compartilhar lógica interna com os "filhos", mantendo-a oculta do "público".

4. Pacote / Default (~)

- **Descrição:** Este nível permite o acesso por qualquer classe que esteja dentro do mesmo pacote (ou *namespace*). Se nenhum modificador for especificado em algumas linguagens (como Java), este é o padrão.

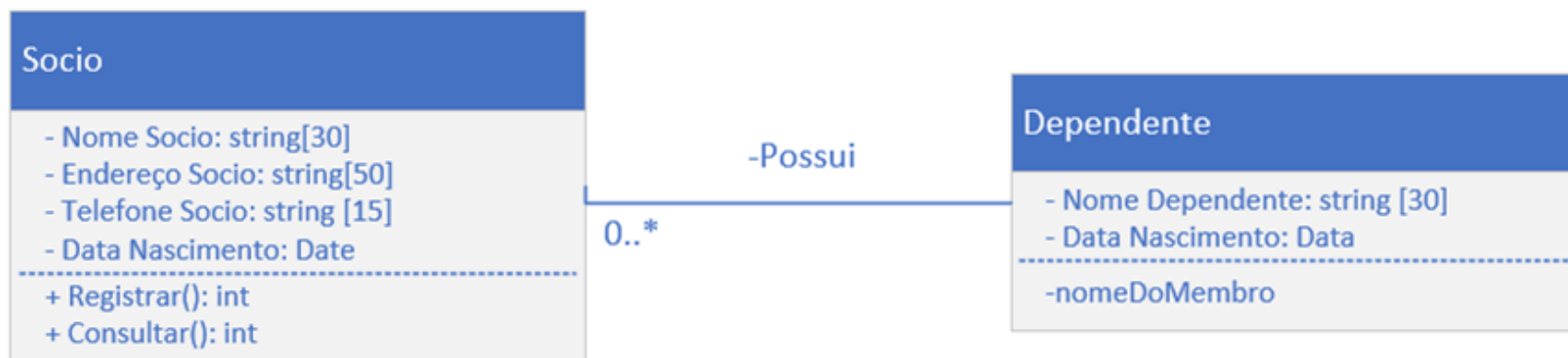
Relacionamentos - Binária

As Classes costumam possuir relacionamento entre si, com o intuito de compartilhar informações e colaborarem umas com as outras para permitir a execução dos diversos processos executados pelo sistema.

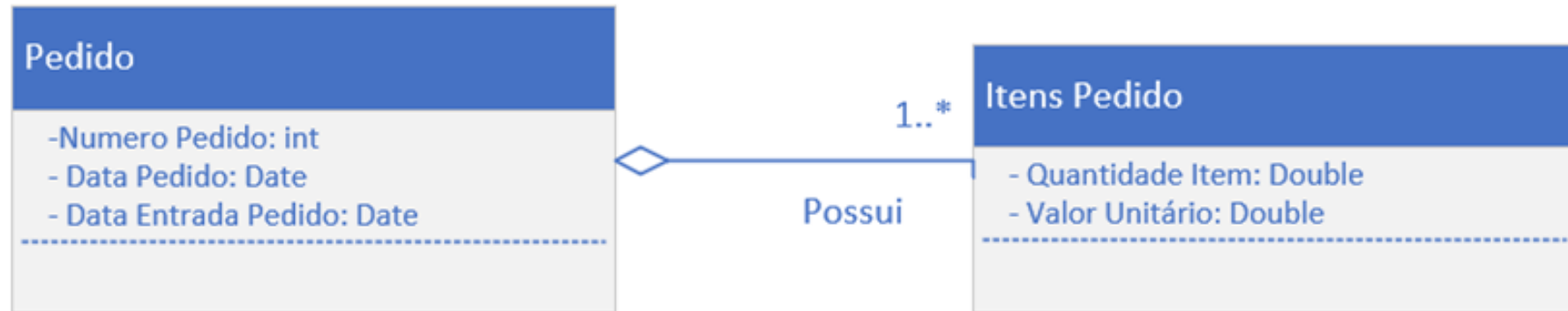
Descreve um vínculo que ocorre normalmente entre duas Classes, chamado neste caso de **Associação Binária**.

Em uma Associação determina-se que as instâncias de uma Classe estão de alguma forma ligadas às instâncias das outras Classes.

Relacionamentos - Binária



Multiplicidade

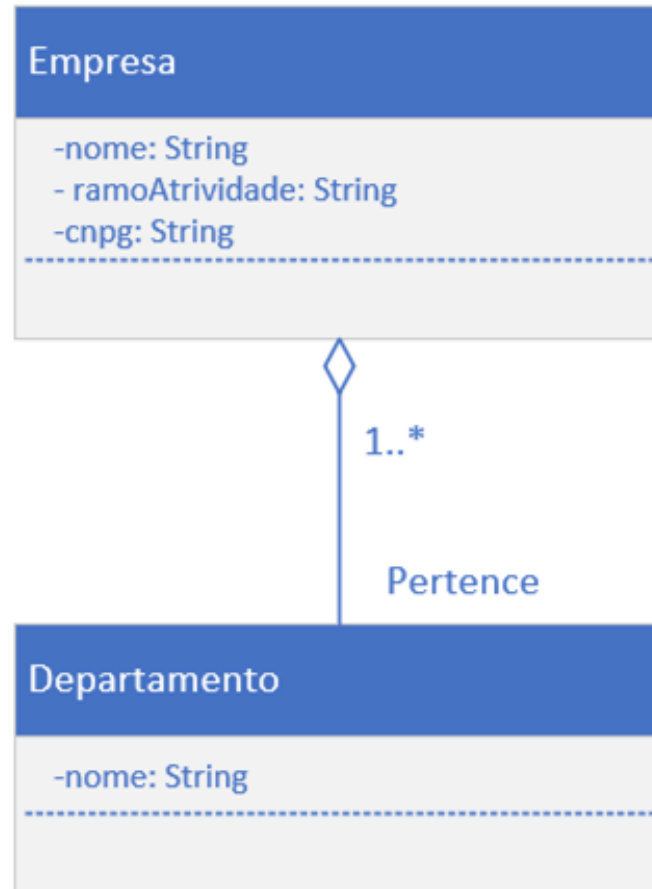


Um pedido pode possuir no mínimo um item como vários itens, e um item de pedido pode pertencer no mínimo a um pedido e no máximo um, 1.1 , já que não foi especificada a multiplicidade

Multiplicidade

0..1	No mínimo zero (nenhum) e no máximo um. Indica que os Objetos da classe associada não precisam obrigatoriamente estar relacionados.
1..1	Um e somente um. Indica que apenas um objeto da classe se relaciona com os objetos da outra classe.
0..*	No mínimo nenhum e no máximo muitos. Indica que pode não haver não instâncias da classe participando do relacionamento.
*	Muitos. Indica que muitos objetos da Classe estão envolvidos no Relacionamento.
1..*	No mínimo um e no máximo muitos. Indica que há pelo menos um objeto envolvido no relacionamento, podendo haver muitos.
3..5	No mínimo 3 e no máximo 5. Indica que há pelo menos 3 instâncias envolvidas no relacionamento e que pode ser 4 ou 5 as instâncias envolvidas, mas não mais do que isso.

Agregação

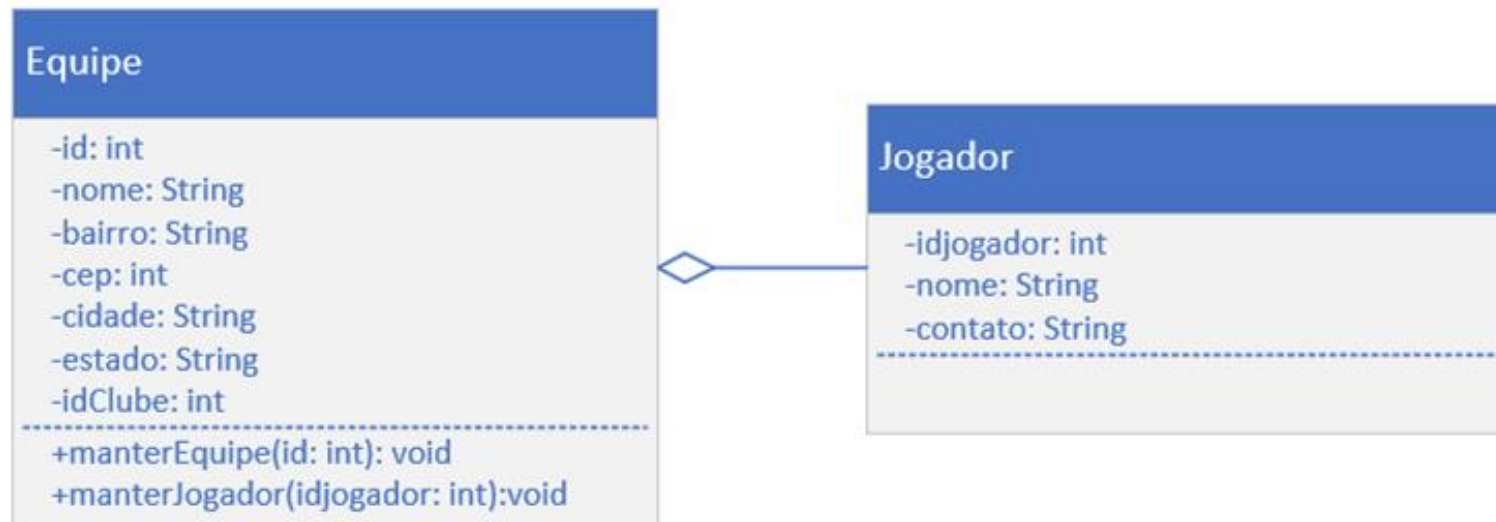


É um tipo especial de associação onde tenta-se demonstrar que as informações de um objeto (chamado objeto-todo) precisam ser complementadas pelas informações contidas em um objeto de outra classe (chamado objeto-parte).

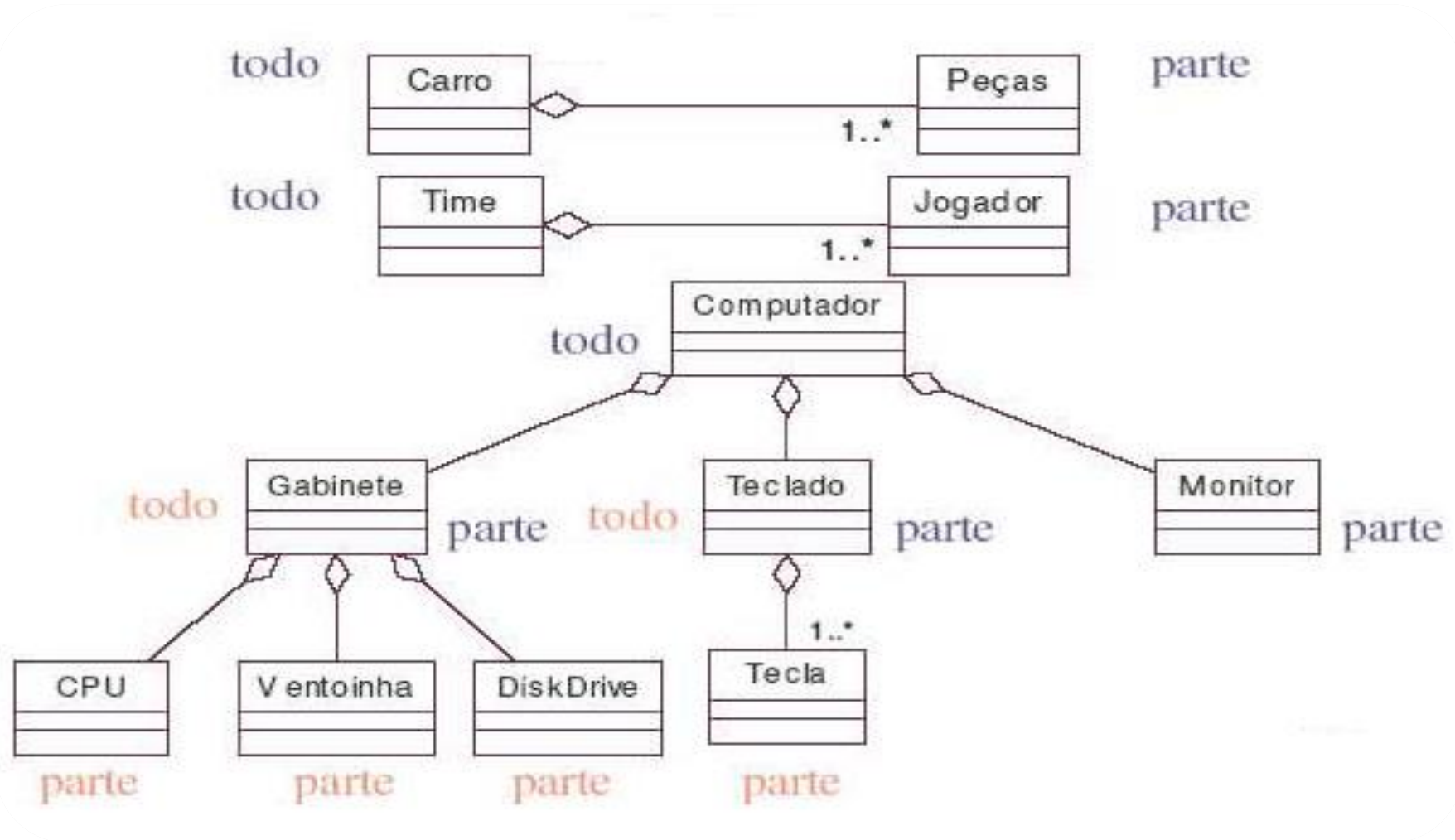
O símbolo de agregação difere do símbolo da associação por conter um losango na extremidade da classe que contém os *objetos-todo*.

Agregação

Tipo de relacionamento que não força a destruição do conjunto, ou seja, uma vez destruído o objeto todo, não há obrigatoriedade da destruição do objeto parte, assim, no exemplo abaixo, mesmo se uma equipe “X” acabar, o jogador poderá fazer parte de outra equipe.



Exemplo

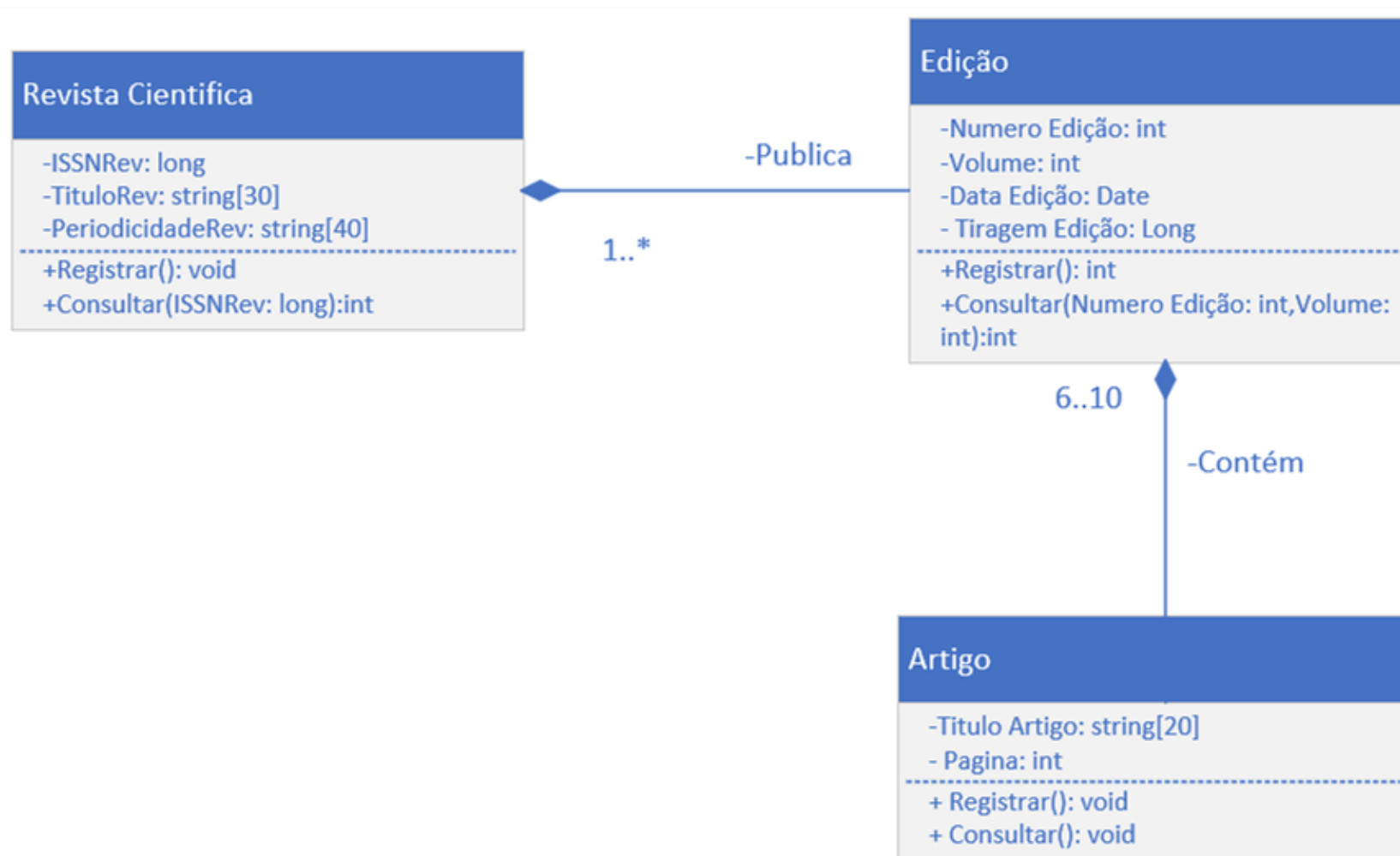


Composição



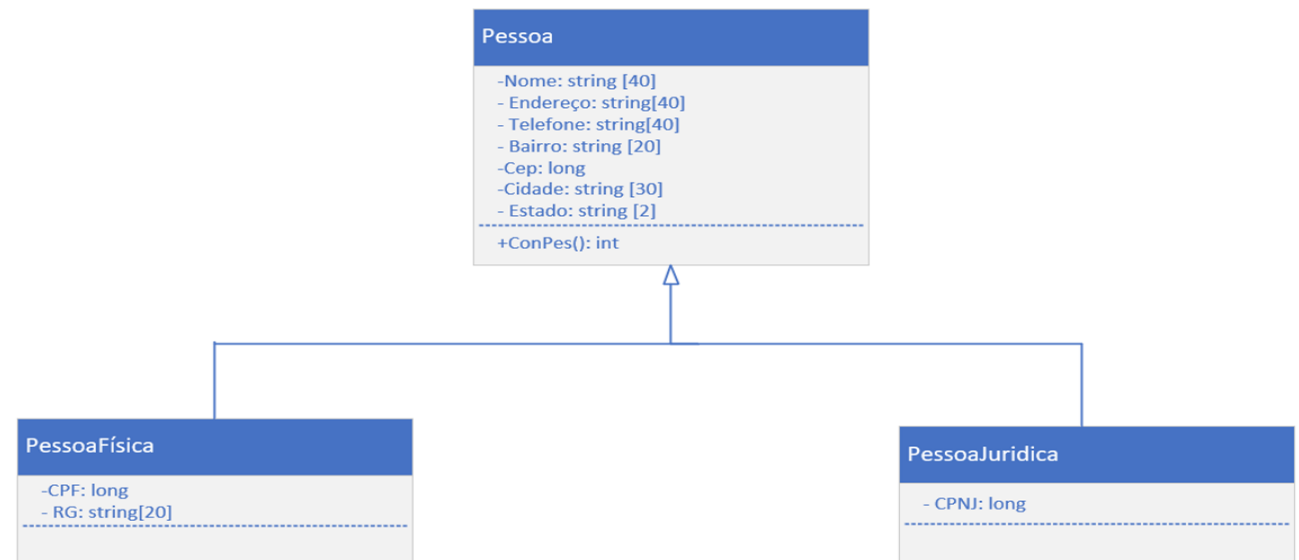
- ✓ Constitui-se em uma variação do tipo agregação.
- ✓ Uma associação do tipo Composição tenta representar um vínculo mais forte entre os objetos-todo e objetos-parte.
- ✓ Tenta mostrar que os objetos-parte têm que pertencer exclusivamente a um único objeto-todo.
- ✓ Só faz sentido existir a parte se houver o todo
- ✓ O símbolo usado para a associação de Composição é um losango preenchido, e da mesma forma que na Agregação, deve ficar ao lado do objeto-todo.

Composição



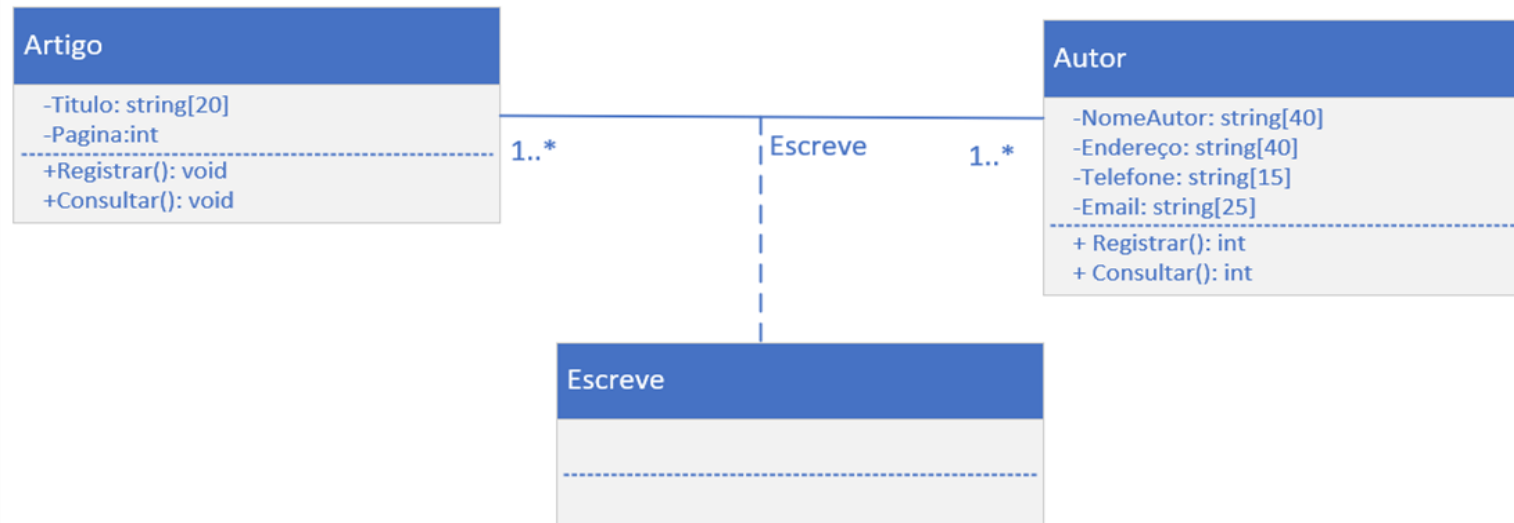
Herança

- ✓ Similar à associação de mesmo nome utilizado no Diagrama de Casos de Uso. Seu objetivo é identificar classes-mãe (gerais) e classes filhas (especializadas).
- ✓ Permite também demonstrar a ocorrência de métodos polimórficos nas classes especializadas.



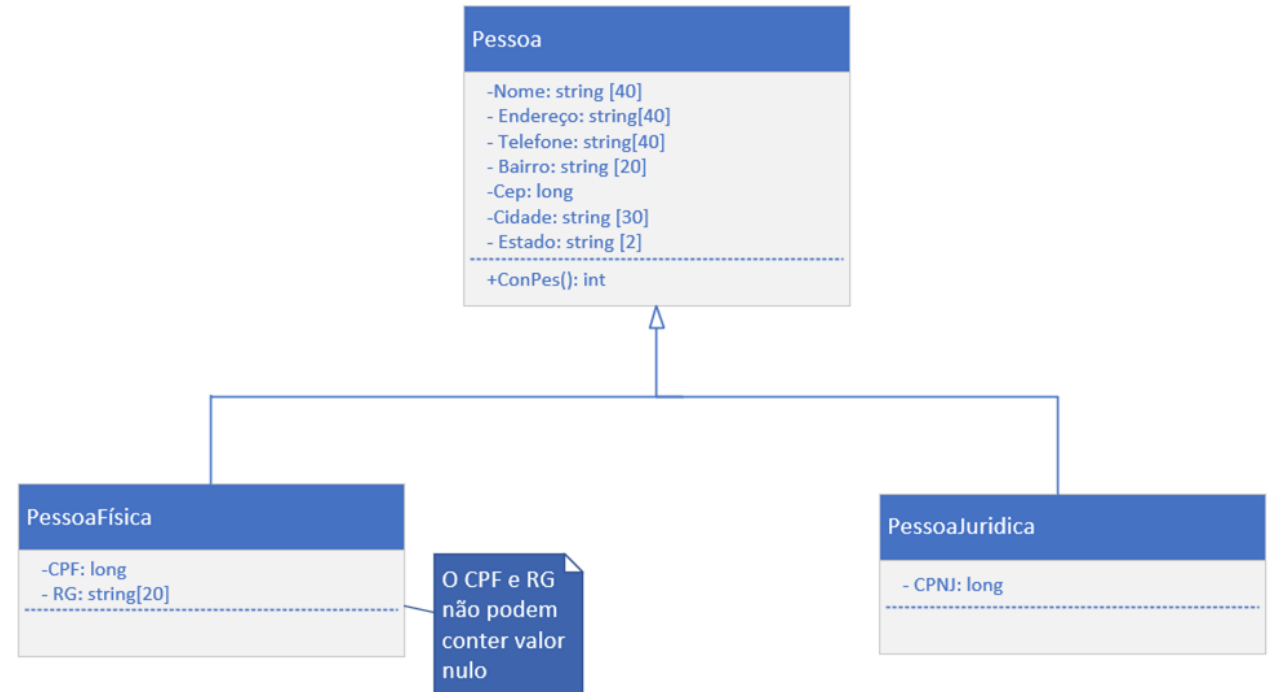
Classes Associativas

- ✓ São classes produzidas quando da ocorrência de associações que possuem multiplicidade muitos (*) em todas as suas extremidades.
- ✓ As Classes Associativas são necessárias nesses casos porque não existe um repositório que possa armazenar as informações produzidas pelas as associações.



Notas

São importantes para informar algum comentário necessário a classe, método ou atributo, fazendo com que, todos tomem conhecimento de forma imediata a observação feita, seja essa observação feita para validar ou simplesmente informar como o objeto notificado se comporta.



Exercícios

Envio: <https://forms.gle/jXgbVDZF7MMnyTsr7>

1) Crie uma classe usando padrão UML

a) Classe: Pessoa

Atributos: nome, idade

Método: CadastrarPessoa()

b) Classe: Porta

Atributos: aberta, cor, dimensaoX, dimensaoY, dimensaoZ

Métodos: abre(), fecha(), pinta(), estaAberta()

2) Crie uma classe conta bancária composta pelos atributos número, nome do Cliente, saldo, limite e com os métodos sacar valor, depositar valor, transferir valor, mostrar titular

Exercícios

3) Identifique as classes e construa os diagramas de classe com os seus atributos para os seguintes contextos:

- a. Numa turma de um curso de graduação, temos disciplinas ministradas em salas diferentes.
- b. Está passando na rede de cinemas ArtFilme o filme “Os vingadores”, todos os dias, em três sessões diárias.

Exercícios

4) Analise os atributos a seguir e determine o nome da classe correspondente, no contexto da informatização de uma clínica médica, monte a estrutura UML

a. Classe: _____

Atributos: nome, CRM, especialidade, data de admissão

Métodos: Realizar Consulta, Prescrever Exame, Prescrever Receita

b. Classe: _____

Atributos: dia, hora, identificador de consulta paga, identificador de comparecimento de cliente

Operações: Marcar, Desmarcar, Remarcar, Pagar Consulta, Registrar Comparecimento do Paciente

Exercícios

5) Seguindo a abordagem de orientação a objetos, identificar no enunciado abaixo os objetos e usuários do sistema. Liste os nomes dos objetos, seus atributos e os usuários do sistema

UMA LOCADORA de veículos necessita de um sistema para facilitar o atendimento a seus clientes. Os carros são classificados por tipo: popular, luxo e utilitário.

As informações que interessam à locadora sobre cada um dos veículos são: placa do carro, valor diário do aluguel.

Os funcionários da locadora são responsáveis pelo cadastro dos clientes e dos veículos.

Eles também fazem as locações e encerram as mesmas. Há clientes especiais e comuns.

Os especiais têm direito a uma taxa de desconto e um valor de quilometragem extra nas suas locações. Um cliente é identificado pelo nome, número do cartão de crédito e data de expiração.